

A Hierarchical Deep Reinforcement Learning Framework for 6-DOF UCAV Air-to-Air Combat

Jiajun Chai¹, Graduate Student Member, IEEE, Wenzhang Chen¹, Yuanheng Zhu¹, Senior Member, IEEE, Zong-Xin Yao, and Dongbin Zhao², Fellow, IEEE

Abstract—Unmanned combat air vehicle (UCAV) combat is a challenging scenario with high-dimensional continuous state and action space and highly nonlinear dynamics. In this article, we propose a general hierarchical framework to resolve the within-vision-range (WVR) air-to-air combat problem under six dimensions of degree (6-DOF) dynamics. The core idea is to divide the whole decision-making process into two loops and use reinforcement learning (RL) to solve them separately. The outer loop uses a combat policy to decide the macro command according to the current combat situation. Then the inner loop uses a control policy to answer the macro command by calculating the actual input signals for the aircraft. We design the Markov decision-making process for the control policy and the Markov game between two aircraft. We present a two-stage training mechanism. For the control policy, we design an effective reward function to accurately track various macro behaviors. For the combat policy, we present a fictitious self-play mechanism to improve the combat performance by combating against the historical combat policies. Experiment results show that the control policy can achieve better tracking performance than conventional methods. The fictitious self-play mechanism can learn competitive combat policy, which can achieve high winning rates against conventional methods.

Index Terms—6-DOF unmanned combat air vehicle (UCAV), air combat, hierarchical structure, reinforcement learning (RL), self-play.

I. INTRODUCTION

UNMANNED combat air vehicle (UCAV), an aircraft that can carry missiles for combat, is becoming more and

more important in the future battlefield [1]. Compared with manned aircraft, UCAV can greatly reduce the volume and weight of aircraft because it is not limited by the physical endurance of human pilots. The resulting lower cost and greater maneuverability make it an important member of the modern war system [2]. Air-to-air combat is a critical application scenario of UCAV, which can be categorized into beyond visual range (BVR) combat and within visual range (WVR) combat according to the combat range [3]. This study focuses on WVR combat, in which the aircraft fights against the opponent at close range. More specifically, an aircraft has a weapon engagement zone (WEZ) in front of it as the attack range of the aircraft guns or short-range missiles. If the aircraft remains within its opponent's WEZ for a duration, it will be shot down. To achieve victory in combat, each aircraft must attack within its WEZ and evade opponent attacks by executing appropriate maneuvers.

Intelligent air combat has attracted a lot of attention. The goal is to control the aircraft autonomously to shoot down the opponent aircraft without being attacked. It is necessary to achieve unmanned control of the aircraft to save the lives of pilots [4]. Although the UCAV can be controlled remotely by humans, there is a risk of losing contact due to electronic interference [5]. Therefore, it is critical to provide autonomous control for UCAV through artificial intelligence methods. There have been studies trying to achieve intelligent air combat since the 1950s [6]. Early works focused on imitating the behavior of human pilots. Some methods are based on several control rules [7], [8] or fuzzy logic [9] to realize the experts experience. These methods are limited by the difficulty in exhaustively enumerating expert behavior, which constrains their applicability.

In order to provide intelligence for aircraft without expert experience, a lot of works consider learning-based methods to obtain powerful combat policies. McGrew et al. [10] adopted approximate dynamic programming to compute an efficient strategy. Most learning-based methods adopt reinforcement learning (RL) to learn the optimal combat policies by interacting with environments instead of knowing the dynamics in advance [11]. Researchers have successfully applied RL to some decision-making tasks [12], such as Go [13], video games [14], and StarCraft II [15], [16]. Besides, there are also some works that focus on multiagent cooperation [17] and competition [18], [19], [20]. Attracted by

Manuscript received 10 February 2023; accepted 12 April 2023. Date of publication 8 May 2023; date of current version 18 August 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2018AAA0101005; in part by the National Natural Science Foundation of China under Grant 62293541; in part by the Strategic Priority Research Program of Chinese Academy of Sciences under Grant XDA27030400; and in part by the Youth Innovation Promotion Association CAS. This article was recommended by Associate Editor D. Wang. (Corresponding author: Yuanheng Zhu.)

Jiajun Chai, Wenzhang Chen, Yuanheng Zhu, and Dongbin Zhao are with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: yuanheng.zhu@ia.ac.cn).

Zong-Xin Yao is with the Department of Unmanned Aerial Vehicle, Shenyang Aircraft Design and Research Institute, Shenyang 110035, China.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TSMC.2023.3270444>.

Digital Object Identifier 10.1109/TSMC.2023.3270444

the performance of RL algorithms, there are a lot of works that adopt RL to train powerful combat policies in WVR air combat. Many works adopt deep Q -learning [21] to solve the air combat task in discrete action space [22], [23], [24]. Yuan et al. [25] proposed a novel heuristic deep deterministic policy gradient (DDPG) algorithm to improve the exploration efficiency of the UCAV combat policy in continuous action space. Li et al. [26] adopted proximal policy optimization (PPO) to train the policy fighting against a min-max policy. Due to the model-free property of RL, there is no need for prior analysis or identification of the aircraft dynamics like in traditional works [27]. In addition, some works propose game-theoretic methods to solve the air combat task. Park et al. [28] proposed a differential game-based scoring function to search the optimal maneuvers. Wang et al. [29] established a policy pool and adopt deep Q -learning with alternate freeze game to improve the combat policy. Kong et al. [30] combined multiagent DDPG with game theory in one-on-one air combat. The aforementioned methods assume 3-DOF aircraft dynamics for the air combat task, which neglects the attack and sideslip angles by assuming that the velocity vector aligns with the nose direction. This oversimplification renders these methods unsuitable for more complex and realistic 6-DOF aircraft dynamics.

The hierarchical framework is used to decrease training difficulty [31]. Due to the greater difficulty in directly training on 6-DOF dynamics compared to 3-DOF dynamics, researchers have utilized hierarchical frameworks to divide the decision-making process into two loops [4], [32], [33]. The outer loop combat policy sends macro commands to the inner loop based on the relative observation of the opponent aircraft as well as the condition of the ego aircraft. The inner loop control policy manipulates the aircraft directly to answer the macro commands from the combat policy. Shin et al. [4] and Wang et al. [32] used nonlinear inverse control (NDI)-based control policy to implement the inner loop. For the outer loop, [32] uses the approximate dynamic programming to get the combat policy, and [4] answers the commands from predefined basic fighter maneuvers. The NDI-based controller [34] can be replaced by the error-based [35], [36], event-triggered [37], and fault-tolerant [38] control policies. However, most of the existing control policies are studied on simple tasks, such as landing [39], lateral control, or longitudinal control [40], and may not be effective for more complex maneuvers required in air combat.

Pope et al. [41] designed a different hierarchical framework and train the inner and outer loop only by RL. The inner loop consists of the aircraft and a subpolicy, which is trained by soft actor-critic (SAC) [42]. To enhance maneuvering performance with different preferences, the researchers modified the reward function to train various subpolicies. The policy selector in the outer loop, trained by SAC, chooses an appropriate subpolicy for the current situation. The fictitious self-play mechanism improves policy generation iteratively. This policy eventually ranked second in the AlphaDogfight Trials and defeated a human expert. Nevertheless, designing well-performing reward functions for subpolicy training remains challenging. Additionally, the decision model's neural

network demands significant resources, with tens of thousands of neurons per layer.

A. Contribution

In this article, we focus on WVR air-to-air combat under 6-DOF dynamics. Our goal is to present a powerful and model-free policy under limited computing resources. We adopt the hierarchical framework to make decisions for the aircraft and divide the decision-making process into the outer loop and the inner loop, which maintain a combat policy and a control policy, respectively. The main contributions are threefold.

- 1) We propose a hierarchical framework that consists of two policies: a) a combat policy to determine target roll and pitch angles based on the current combat situation, and b) a control policy to generate actual input signals for the aircraft to track these angles. Compared with other methods, our framework is entirely model free and offers superior combat performance.
- 2) We present an effective reward function by comprehensively balancing the control performance and aircraft constraints for the PPO algorithm with detailed analysis. Compared with traditional control methods, it can achieve better tracking performance in a wider range of scenarios without access to the aircraft dynamics.
- 3) We design a fictitious self-play mechanism to improve the performance of the combat policy by fighting against the historical combat policies generation by generation. In each generation, we find the best response of the historical combat policies and store it in a pool. Experiment results show that each generation of combat policies can achieve better combat performance against the former policies, and the final policy can get a high winning rate against traditional methods.

B. Organization

This article is organized as follows. Section II introduces the problem formulation of our air combat task and the PPO algorithm. Section III first introduces the overall hierarchical framework of our method. Sections IV and V describe the control policy and the combat policy in detail, respectively. Sections VI and VII show the experiments and their results. Finally, Section VIII gives the conclusion.

II. PROBLEM FORMULATION

A. Air-Combat Scenario

The first part of Table I shows the major state variables of a 6-DOF aircraft. The motion of the 6-DOF aircraft can be split into translational and rotational motions. For the translational motion, the aircraft changes its coordinates in the three dimensional (3-D) space, which are northward displacement pn , eastward displacement pe , and altitude alt , respectively. The velocity vector shown in Fig. 1 indicates the translational motion. The velocity of aircraft vt is the 2-norm of the velocity vector

$$vt = \sqrt{\dot{pn}^2 + \dot{pe}^2 + \dot{alt}^2} \quad (1)$$

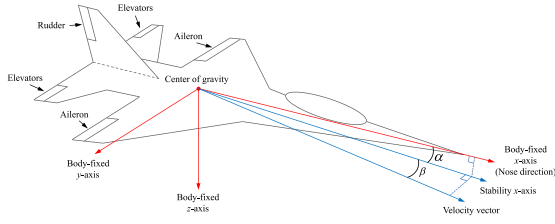


Fig. 1. Major components of 6-DOF dynamics [27].

TABLE I
STATE AND ACTION VARIABLES OF THE 6-DOF AIRCRAFT [33]

Variable	Bound	Meaning
v_t	[0 m/s, 1.2 * 340 m/s]	Aircraft velocity
pn	-	Northward displacement
pe	-	Eastward displacement
alt	[0 m, 12000 m]	Aircraft altitude
ϕ, θ, ψ	$[-\pi, \pi]$	Roll, pitch, and yaw angles
P, Q, R	-	Roll, pitch, and yaw rate
α	$[-\pi/6, \pi/3]$	Attack angle
β	$[-\pi/6, \pi/6]$	Sideslip angle
$power$	-	Engine power lag
δ_{th}	[0, 1]	Actual engine throttle
δ_a	$[-\pi/9, \pi/9]$	Actual aileron angle
δ_e	$[-5\pi/36, 5\pi/36]$	Actual elevator angle
δ_r	$[-\pi/6, \pi/6]$	Actual rudder angle
δ_{th}^{cd}	[0, 1]	Engine throttle command
δ_a^{cd}	$[-\pi/9, \pi/9]$	Aileron angle command
δ_e^{cd}	$[-5\pi/36, 5\pi/36]$	Elevator angle command
δ_r^{cd}	$[-\pi/6, \pi/6]$	Rudder angle command

where $\dot{pn}$, $\dot{pe}$, and $\dot{alt}$ are the derivatives of pn , pe , and alt with respect to time, respectively. Besides, for the rotational motion, the Euler angles can represent the relationship between the aircraft body-fixed coordinate system and the ground coordinate system. The roll, pitch, and yaw angles are represented by ϕ , θ , and ψ , respectively, and P , Q , and R are their derivatives of time.

We illustrate the meaning of some state variables in Fig. 1. The attack angle α is the angle between the nose direction and the stability x -axis, which is the projection of the velocity vector on the body-fixed X - Y plane. The sideslip angle β is the angle between stability x -axis and velocity vector. The motion of an aircraft is affected by the combined effects of gravity, aerodynamics, and engine power. For the engine power term, the throttle of the engine δ_{th} determines the acceleration in the body-fixed x -axis. $power$ is the intermediate invariable of δ_{th} and represents the actual power of the engine. The aircraft has three control surfaces for the aerodynamics term. δ_a , δ_e , and δ_r are the deflection angles of the aileron, elevator, and rudder, which are used to control the roll, pitch, and yaw of the aircraft, respectively. The second part of Table I shows the action variables of the 6-DOF aircraft. The commands $u = \{\delta_{th}^{cd}, \delta_a^{cd}, \delta_e^{cd}, \delta_r^{cd}\}$ specifies the desired control signals that drive the current deflection angles of the aircraft to the commanded values. The state of an aircraft is governed by 12 nonlinear equations of motion [33]. The dynamics can be

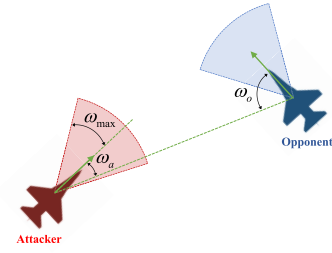


Fig. 2. Attack range of each aircraft in the combat scenario.

simplified as

$$\dot{x} = f(x, u) \quad (2)$$

where $x = [vt, alt, \alpha, \beta, \phi, \theta, P, Q, R, pow, \delta_a, \delta_e, \delta_r]$ is the state of an aircraft, u is the commands to the aircraft, $f(\cdot)$ is the nonlinear dynamic function for the aircraft motion. The details of the dynamics have been summarized by [33], [43], and [44].

The one-on-one air combat task has a zero-sum setting, because the goal of each aircraft is to shoot down its opponent without being attacked. In this work, the battlefield is composed of two 6-DOF aircraft, each of which carries infrared homing missiles to attack opponents. As shown in Fig. 2, the WEZ of the aircraft is a conical area. If an aircraft enters the WEZ of its opponent, it will be locked by the opponent's missile and shot down. The cone angle ω_{max} of WEZ is 30° , and the generating line is 3 km. ω_a is the antenna train angle (ATA), which is the angle between the line-of-sight (LOS) vector and the attacker aircraft nose direction. ω_o is the aspect angle (AA), which is the angle between the LOS vector and the opponent aircraft nose direction.

B. Proximal Policy Optimization

RL aims to get the optimal policy of the agent through the interaction with the environment. At each step of the interaction, the agent gets a state s_k from the environment and determines an action a_k for the state transition, where k is the timestep. Then the environment will provide a reward r_{k+1} to the agent to evaluate the action just being executed. The aim of RL is to maximize the sum of discounted rewards, which is also called return

$$G_k = r_{k+1} + \gamma r_{k+2} + \gamma^2 r_{k+3} + \dots \quad (3)$$

where $\gamma \in [0, 1]$ is the discount factor that indicates history impact.

PPO is a widely-used RL algorithm based on policy gradient [45]. It maintains two important modules: 1) actor and 2) critic, and both of them are generally represented by neural networks. The actor indicates the policy function $a \sim \pi(s; \Theta)$, which maps from states to action distribution probabilities. The critic $V(s; \Phi)$ approximates the state value function $V_\pi(s) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{k+1} | s_0 = s]$, which is the expected return under policy π . Θ and Φ are parameters of two neural networks, respectively. The agent interacts with the environment and stores $\{s_k, a_k, r_k\}$ into a buffer. At the end of each episode, the returns $\{G_k\}_{k=0}^K$ of this episode are calculated

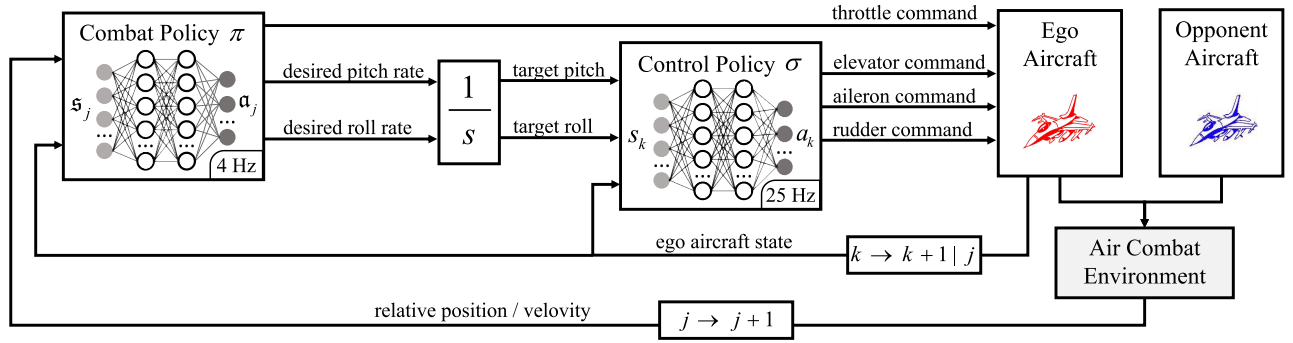


Fig. 3. Hierarchical control framework for 6-DOF aircraft.

according to (3) and stored into the buffer. The advantage value $A_k(s_k, a_k)$ is calculated by generalized advantage estimator (GAE) [46] and also stored into the buffer

$$\hat{A}_k = \eta_k + (\gamma\lambda)\eta_{k+1} + \dots + (\gamma\lambda)^{K-k-1}\eta_{K-1}$$

where $\eta_k = r_{k+1} + \gamma V(s_{k+1}; \Phi) - V(s_k; \Phi)$ (4)

where $\hat{A}_k$ is the estimated value of $A_k(s_k, a_k)$, λ is the hyperparameter of GAE, Φ is the critic parameter, and K is the timestep at the end of an episode. The advantage value is used to evaluate how much taking a_k is better than the other actions. The PPO algorithm uses the stored data to optimize the critic and actor. The loss function of the critic is the mean squared error

$$L_{\text{critic}}(\Phi) = \mathbb{E}_k[V(s_k; \Phi) - G_k]^2. \quad (5)$$

The learning objective of actor is maximizing the following objective function:

$$L_{\text{actor}}(\Theta) = \mathbb{E}_k \left[\min \left(\rho_k(s_k, a_k; \Theta) \hat{A}_k, \text{clip}[\rho_k(s_k, a_k; \Theta), 1 - \varepsilon, 1 + \varepsilon] \hat{A}_k \right) \right] \quad (6)$$

where $\rho_k(s_k, a_k; \Theta) = \pi(a_k|s_k; \Theta)/\pi(a_k|s_k; \Theta_{\text{old}})$ is the probability ratio that measures the difference between $\pi(\Theta)$ and $\pi(\Theta_{\text{old}})$, where Θ_{old} is the actor parameters when sampling these data. This objective is modified from the surrogate objective $\mathbb{E}_k[\rho_k(a_k|s_k; \Theta)\hat{A}_k]$. Maximizing this surrogate objective may cause an excessively large policy update, making it difficult to accurately evaluate the updated policy on old data. Therefore, the objective uses the clip operation in (6) to avoid this by limiting the ρ_k of new policy. ε is the hyperparameter to adjust the boundedness of the constraint. By minimizing (5) and maximizing (6), PPO can improve policy performance and approach optimal performance.

III. HIERARCHICAL FRAMEWORK FOR 6-DOF AIR COMBAT

In the 6-DOF air combat task, it is difficult to make decisions end to end due to the high-dimensional state and action space and the highly nonlinear aircraft dynamics. In this work, we propose a hierarchical framework to divide the decision-making process into two loops as shown in Fig. 3. The outer loop considers the combat situation to decide macro commands, and the inner loop manipulates the aircraft directly

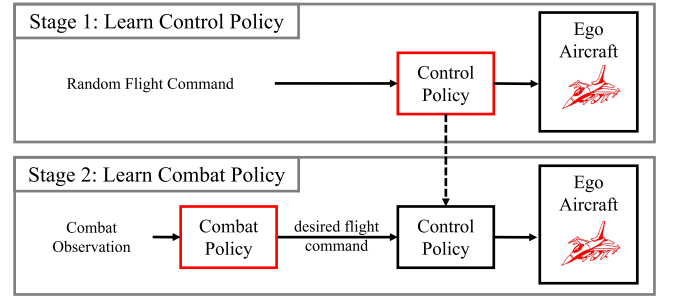


Fig. 4. Two-stage training mechanism. Red box represents the policy being optimized at current stage.

to answer the commands. The two loops run at different frequencies.

The inner loop control policy σ is used to control the aircraft to achieve the desired attitude. It receives the target attitude and the aircraft state and outputs three deflection angle commands for the aircraft attitude control. Compared with traditional methods, the RL-based control policy can achieve better performance in a wider range of situations. The control policy runs at a higher frequency (25 Hz by default) to adjust the aircraft attitude promptly.

The outer loop combat policy π utilizes the inner-loop control policy to manipulate the aircraft and aims to defeat the opponent. It outputs the desired attitude according to the aircraft state and the relative position and velocity of its opponent. With the help of control policy, the combat policy only needs to determine the high-level behaviors, rather than directly determines the raw input signals for the aircraft. The combat policy runs at a lower frequency (4 Hz by default) to keep the aircraft from becoming unstable due to frequent changes in macro commands.

We present a two-stage training mechanism shown in Fig. 4 to stabilize the training process. At stage 1, we use the PPO algorithm to train the control policy to answer random flight commands. At stage 2, we use the trained control policy and adopt the fictitious self-play mechanism to train the combat policy. The details are described in the following two sections. The advantage of our method is that the hierarchical framework reduces the decision-making difficulty, and the two-stage training mechanism improves the training efficiency.

IV. LEARNING THE 6-DOF CONTROL POLICY

In this section, we describe our RL-based control policy in detail. It achieves longitudinal and lateral control by controlling the pitch and roll angles of the aircraft. Since the three Euler angles are coupled, the yaw angle can be controlled indirectly through pitch and roll angles, as described in previous studies [33]. In more detail, the control policy receives the target *pitch* and *roll* angles as tracking targets and outputs deflection angle commands of the aileron, elevator, and rudder to control the aircraft directly. The control process of the inner loop can be established as a Markov decision process (MDP): $U = \{S, A, R, T, \gamma_{in}\}$, where S is state space, A is action space, R is reward function, T is state transition, and γ_{in} is the discount factor. The details are as follows.

- 1) *State*: The state s_k for the control policy at timestep k describes the current situation of controlling the aircraft. It can be divided into two parts. The first part is the *aircraft state* x_k , which has been introduced in Section II-A. It contains enough variables to describe the aircraft's current state for attitude control. The second part describes the current situation of tracking the target pitch and roll angles. It contains the tracking error of pitch angle and roll angle: $[\tilde{\theta}_k - \theta_k, \tilde{\varphi}_k - \varphi_k]$, where $\tilde{\theta}_k$ and $\tilde{\varphi}_k$ are the target pitch angle and target roll angle, respectively. In conclusion, state s_k is $[x_k, \tilde{\theta}_k - \theta_k, \tilde{\varphi}_k - \varphi_k]$. All bounded variables are normalized according to their upper and lower bounds.
- 2) *Action*: The action a_k contains the deflection angle commands of aileron δ_a^{cd} , elevator δ_e^{cd} , and rudder δ_r^{cd} , which can control the roll, pitch, and yaw of the aircraft, respectively.
- 3) *Reward*: The reward is one of the most important elements in the RL training process. For the training of the control policy, we design an effective reward function as follows:

$$r_k = r_k^{\text{alive}} + c_1 r_k^{\text{track}} + c_2 r_k^{\text{action}}. \quad (7)$$

The first part r_k^{alive} is an alive bonus. Its assignment rule is as follows:

$$r_k^{\text{alive}} = \begin{cases} +1, & \text{alive} \\ 0, & \text{instability or crash.} \end{cases} \quad (8)$$

If the aircraft does not fail due to instability or crash, it will receive a positive reward. The instability is related to the attack angle α and sideslip angle β , and the crash is related to the altitude. If any of them exceed their limit values given in Table I, it will be considered as instability or a crash. The second part r_k^{track} is the tracking reward, which is the negative mean square error between the target angles and the current pitch and roll angles

$$r_k^{\text{track}} = -\sqrt{(\tilde{\theta}_k - \theta_k)^2 + (\tilde{\varphi}_k - \varphi_k)^2} \quad (9)$$

where $\tilde{\theta}_k$ and $\tilde{\varphi}_k$ are the target pitch and roll angles, respectively. The third part r_k^{action} is the action reward that is helpful to generate smooth control signals and reduce system fluctuation. It is the negative square sum

Algorithm 1: PPO-Based Control Policy Training Algorithm

```

1 Initialize parameters of actor  $\Theta_{in}$  and critic  $\Phi_{in}$ ;
2 for  $epoch = 1, 2, 3, \dots$  do
3   Initialize the replay buffer  $\mathcal{M}_{in}$ ;
4   # Simulating
5   Use  $\sigma(\cdot; \Theta_{in})$  to follow random target signals and
   store the experience in  $\mathcal{M}_{in}$  until  $\mathcal{M}_{in}$  is full;
6   # Updating
7   Update  $\Theta_{in}$  and  $\Phi_{in}$  based on  $\mathcal{M}_{in}$ ;
8 end

```

of actions to encourage the control policy to control the aircraft smoothly

$$r_k^{\text{action}} = -\sqrt{(\delta_a^{cd})^2 + (\delta_e^{cd})^2 + (\delta_r^{cd})^2}. \quad (10)$$

We can adjust the coefficients c_1 and c_2 of r_k to change the effect of these parts. We will discuss it in the experiment.

- 4) *State Transition*: We use the Runge–Kutta method [44] to implement the discrete-time transition function T , which can be described by $x_{k+1} = \int_{k*T_{in}}^{(k+1)*T_{in}} f(x(\tau), [a_k, \delta_{th}^{cd}]) d\tau$, with T_{in} is the decision interval set to 0.04 s, and δ_{th}^{cd} is the throttle command that is an external variable for this MDP.

The control process with state transition is summarized in the inner loop of Fig. 3. The control policy $\sigma(\cdot|s; \Theta_{in})$ is represented by a neural network with Θ_{in} as its parameter. It can output action $a \sim \sigma(\cdot|s; \Theta_{in})$ based on the state s . The control policy receives the target pitch angle $\tilde{\theta}$ and target roll angle $\tilde{\varphi}$ to calculate the tracking errors, and then combines them with the aircraft state x to form micro state s . Then, the control policy outputs deflection angle commands $a = [\delta_a^{cd}, \delta_e^{cd}, \delta_r^{cd}]$ according to s . Finally, the aircraft translates to the next state s' by executing $[a, \delta_{th}^{cd}]$.

The control policy is trained by the PPO algorithm, and we present the training algorithm in Algorithm 1. The parameters of the actor and critic network for control policy are initialized as Θ_{in} and Φ_{in} . At the beginning of each epoch, we need to initialize a replay buffer $\mathcal{M}_{in}$. In line 5 of Algorithm 1, we start simulating until $\mathcal{M}_{in}$ is full. The random target signal is the sequence of roll and pitch angles. It can be described by the following formula:

$$\tilde{y} = \begin{cases} \text{clip}(y_0 + \dot{y} * (t - t_0), y_0, \mathcal{Y}) & \dot{y} > 0 \\ y_0 & \dot{y} = 0 \\ \text{clip}(y_0 + \dot{y} * (t - t_0), \mathcal{Y}, y_0) & \dot{y} < 0 \end{cases} \quad (11)$$

where $\tilde{y}$ is the target signal, which can be the target pitch angle $\tilde{\theta}$ and the target roll angle $\tilde{\varphi}$. $\dot{y}$ is the angle rate, which ranges from $[-0.2 \text{ rad/s}, 0.4 \text{ rad/s}]$ for pitch angle rate and from $[-0.8 \text{ rad/s}, 0.8 \text{ rad/s}]$ for roll angle rate. $\mathcal{Y}$ is the stable angle, and its range is the range of pitch angle or roll angle shown in Table I. During training, both $\dot{y}$ and $\mathcal{Y}$ are randomly sampled to create target signals. t and t_0 are the current and initial time, respectively. y_0 is the initial angle at t_0 .

The target signal and the external command δ_{th}^{cd} should be reinitialized at a fixed time length to create a diversity of training scenarios. The trajectory data is stored into the PPO buffer $\mathcal{M}_{in}$. When enough data is collected, we update the actor parameters Θ_{in} and critic parameters Φ_{in} according to the loss functions described in Section II-B. By repeating the training algorithm, the PPO-based control policy can converge and achieve good tracking performance, which will be verified in experiments.

V. LEARNING THE COMBAT POLICY

In this section, we train the outer loop combat policy based on the control policy trained in the last section. As shown in Fig. 3, the combat policy outputs the macro commands according to the aircraft state and the relative position and velocity of its opponent. The air combat task can be formulated as a symmetric two-player zero-sum Markov game: $\mathcal{MG} = \{\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma_{ou}\}$. In $\mathcal{MG}$, $\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$ represent the state space, action space, and reward function of player 1, respectively, and $\bar{\mathcal{S}}$, $\bar{\mathcal{A}}$, and $\bar{\mathcal{R}}$ represent those of player 2. Since the air combat is a symmetric zero-sum game, the two player have the same state and action space: $\mathcal{S} = \bar{\mathcal{S}}$, $\mathcal{A} = \bar{\mathcal{A}}$, and the sum of their rewards is always 0: $\mathcal{R} + \bar{\mathcal{R}} = 0$. We introduce the elements in $\mathcal{MG}$ from the perspective of player 1 as follows. To avoid confusion, we use s_j , a_j , and r_j to distinguish from s_k , a_k , and r_k of the control policy. Since the inner loop and outer loop are at different time scales, we use the subscript j to distinguish from k of the control policy.

- 1) *State*: The state s_j comes from the combat scenario and consists of two parts. The first part is the relative position and velocity of the opponent aircraft in the body-fixed coordinate system of the ego aircraft as follows:

$$\begin{aligned} p_{re} &= M[pn - \bar{p}n, pe - \bar{p}e, alt - \bar{a}lt] \\ v_{re} &= M * \bar{v}r[\cos(\bar{\theta})\cos(\bar{\psi}), \cos(\bar{\theta})\sin(\bar{\psi}), \sin(\bar{\theta})] \end{aligned} \quad (12)$$

where the variables with superscript bar indicate that they belong to the opponent aircraft, M is the transformation matrix from the earth coordinate system into the body-fixed coordinate system of the ego aircraft

$$M = \begin{bmatrix} \cos \theta \cos \varphi & \cos \theta \sin \varphi & -\sin \theta \\ -\sin \varphi & \cos \varphi & 0 \\ \sin \theta \cos \varphi & \sin \theta \sin \varphi & \cos \theta \end{bmatrix}. \quad (13)$$

The second part consists of the aircraft state variables $[vt, alt, \alpha, \beta, \varphi, \theta, P, Q, R, pow]$.

- 2) *Action*: The action a_j has two parts. The first part is the throttle command δ_{th}^{cd} that is directly provided to the aircraft. The second part consists of the expected roll rate $\dot{\varphi}$ and pitch rate $\dot{\theta}$. They are used to calculate the target angles (roll $\tilde{\varphi}$ and pitch $\tilde{\theta}$) from the current angles to the bounds as follows:

$$\begin{aligned} \tilde{\theta} &= \text{clip}(\dot{\theta} * (t - t_0) + \theta_0, \theta_{\min}, \theta_{\max}) \\ \tilde{\varphi} &= \text{clip}(\dot{\varphi} * (t - t_0) + \varphi_0, \varphi_{\min}, \varphi_{\max}) \end{aligned} \quad (14)$$

where t is the current time, and t_0 is the initial time when this outer loop starts. By designing the angle rates

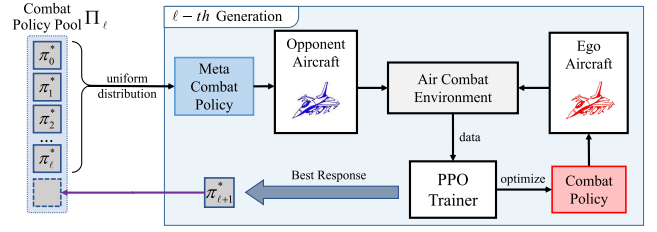


Fig. 5. Fictitious self-play mechanism to improve the performance of combat policy.

as macro actions, the target signals can be smoother, thus reducing the training difficulty. θ_0 and φ_0 are the initial pitch and roll angle at t_0 . $\theta_{\min}$, $\theta_{\max}$, $\varphi_{\min}$, and $\varphi_{\max}$ are the bounds in Table I.

- 3) *Reward*: The reward r_j is sparse, which only occurs at the end of an episode. If the ego aircraft exceeds the altitude limitation or is shot down by the opponent, it will receive a negative penalty, whose value is -100 . If the ego aircraft shoots down the opponent, it will win this combat and receive a positive reward, whose value is $+100$.
- 4) *State Transition*: The transition function $\mathbb{T}$ of the combat policy can be defined as $\Pr(s'|s, a)$. The decision interval of the combat policy is T_{ou} . As shown in Fig. 3, at each decision of the outer loop, the combat policy outputs action a according to state s . Within this outer loop, several inner loops are conducted to track the target angles $\tilde{\theta}$ and $\tilde{\varphi}$ in (14). The macro action a remains unchanged during the execution phase of these inner loops until the next outer loop. At the end of an outer loop, a new state s' is generated and a new outer loop is started. Besides, T_{ou} is a multiple T_{in} to increase the inner-loop flight control stability.

Two players have their own combat policies π and $\bar{\pi}$ to make decisions. The objective for player 1 can be defined as $J(\pi, \bar{\pi}) = \mathbb{E}_{a \sim \pi, \bar{a} \sim \bar{\pi}}[\sum_j r_j]$. It represents the expected value of the sum of rewards obtained by player 1 in each episode when two players use the combat policy combination $(\pi, \bar{\pi})$. The Nash or minimax equilibrium $(\pi^*, \bar{\pi}^*)$ of this zero-sum Markov game is achieved when $J(\pi^*, \bar{\pi}^*) \geq J(\pi, \bar{\pi}^*) \geq J(\pi, \pi^*) \forall \pi, \bar{\pi}$. Furthermore, since $\mathcal{MG}$ is a symmetric game, the condition can be simplified as $J(\pi^*, \pi') \geq J(\pi^*, \pi^*) \geq J(\pi, \pi^*) \forall \pi, \pi'$. The combat policy $a_j \sim \pi(\cdot | s_j; \Theta_{ou})$ is represented by a neural network with Θ_{ou} as its parameter. We adopt the fictitious self-play mechanism, which makes the player optimize its current policy by combating against its historical combat policies, to solve $\mathcal{MG}$. A historical combat policy pool Π is established to record the historical combat policies. The network parameters stored in the pool are always unchanged, and they are uniformly sampled as the combat policies of the opponent aircraft in the fictitious self-play training.

Take the ℓ th generation shown in Fig. 5 as an example, we employ the PPO algorithm to find the best response against Π_ℓ

$$\pi_{\ell+1}^* = \max_{\pi} \sum_{i=0}^{\ell} J(\pi, \pi_i^*) = \mathbf{BR}(\Pi_\ell) \quad (15)$$

Algorithm 2: Training Algorithm of the 6-DOF Combat Policy via Fictitious Self-Play

```

1 Initialize an historical combat policy pool  $\Pi_0$ ;
2 Get a control policy  $\sigma(\cdot; \Theta_{in})$  by Algorithm 1;
3 for generation  $\ell = 1, 2, 3, \dots$  do
4   # Initialize for a New Generation
5   Initialize parameters of actor  $\Theta_{ou}$  and critic  $\Phi_{ou}$ ;
6   # RL Training
7   for epoch  $= 1, 2, 3, \dots$  do
8     Initialize the replay buffer  $\mathcal{M}_{ou}$ ;
9     # Simulating
10    Use  $\pi(\cdot; \Theta_{ou})$  to combat against the combat
    policies sampled randomly from  $\Pi_\ell$  and store the
    experience in  $\mathcal{M}_{ou}$ ;
11    # Updating
12    Update  $\Theta_{ou}$  and  $\Phi_{ou}$  based on  $\mathcal{M}_{ou}$ ;
13  end
14  # Update Pool
15  Store the converged policy in the pool
   $\Pi_{\ell+1} = \Pi_\ell \cup \{\pi(\cdot; \Theta_{ou})\}$ ;
16 end

```

where Π_ℓ is the historical combat policy pool of ℓ th generation, $\mathbf{BR}(\Pi_\ell)$ is the best response against the uniform sampling of Π_ℓ . The training process is shown in Algorithm 2. It has been demonstrated that the algorithm based on fictitious play can converge to Nash equilibrium [47]. It should be noted that we only train the combat policy for player 1 because $\mathcal{M}\mathcal{G}$ is symmetric. The decision making of player 2 is implemented by a meta combat policy shown in Fig. 5. At the beginning of each episode, the meta combat policy samples a combat policy from Π_ℓ with equal probability

$$\pi_{-i} \sim \text{uniform}(\Pi_\ell) \quad (16)$$

where $\text{uniform}(\cdot)$ represents a uniform distribution. The sampling process aims to help the replay buffer $\mathcal{M}_{ou}$ cover enough combat data between the current combat policy and all policies in Π_ℓ . The best response $\pi_{\ell+1}^* = \mathbf{BR}(\Pi_\ell)$ trained by PPO is the policy that can get the highest reward against the uniformly sampled historical combat policy pool. At the end of a generation, $\pi_{\ell+1}^*$ is added to Π_ℓ to form a new historical combat policy pool, and a new generation will start

$$\Pi_{\ell+1} = \Pi_\ell \cup \{\pi_{\ell+1}^*\}. \quad (17)$$

With the fictitious self-play training, the latest policy can achieve better combat performance than the historical combat policies and approach the equilibrium policy.

VI. EXPERIMENTS ON RL-BASED CONTROL POLICY

A. Experimental Setup

In this article, we choose the F-16 as the combat aircraft, because its dynamics and aerodynamic parameters are all open source [44]. It should be noted that the dynamics introduced above is general for all similar 6-DOF aircraft dynamics, instead of the specific model of F-16.

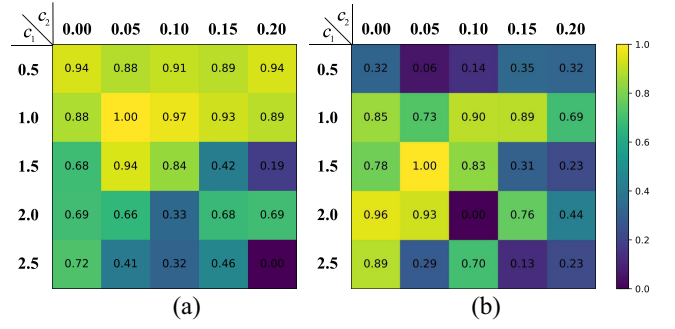


Fig. 6. Performance matrix of control policy. (a) Alive bonus. The control policy with higher score has better ability to keep stability. (b) Tracking performance. The control policy with higher score can track target signals better.

In the experiments on control policy, we set the decision interval T_{in} of the control policy to 0.04 s. For the RL training experiment, we apply the PPO algorithm to optimize the control policy. Since the three parts of the inner loop reward have different effects on the performance of the control policy, we first investigate the effects and find the best control policy. As shown in Fig. 6, since only the relative proportion of three coefficients will affect the performance, we adjust c_1 and c_2 in the experiment. The coefficient c_1 of the tracking penalty ranges from 0.5 to 2.5, and the coefficient c_2 of the action penalty ranges from 0.0 to 0.2. In the experiments of testing the tracking performance of control policies, we initialize the aircraft to level flight state. We test the performance of control policies under three target signals: 1) sinusoidal signal; 2) step signal; and 3) random signal in the form of (11). They can test the tracking performance of the control policy under different conditions.

For comparison, we provide two traditional control policies to track the target roll and pitch angles. The first is the proportional integral derivative (PID) control policy. It is a model-free control policy that outputs commands based on tracking error [35]. Its control signal is shown as follows:

$$\begin{aligned} \delta_e^{cd}(t) &= 0.65 * e_\theta(t) + 0.08 * \int_0^t e_\theta(\tau) d\tau + 25.0 * \frac{de_\theta(t)}{dt} \\ \delta_a^{cd}(t) &= 0.2 * e_\varphi(t) + 0.015 * \int_0^t e_\varphi(\tau) d\tau + 12.0 * \frac{de_\varphi(t)}{dt} \end{aligned} \quad (18)$$

where $e_\theta(t) = \tilde{\theta}(t) - \theta(t)$ and $e_\varphi(t) = \tilde{\varphi}(t) - \varphi(t)$ is the tracking error of pitch and roll angles, respectively. The coefficients in (18) are fine-tuned according to empirical experience to achieve satisfying results. We ignore the deflection angle of the rudder to a fixed 0 because its effect is small.

The second is the linear quadratic regulator (LQR) control policy. It is a model-based control policy that requires the linear model of the aircraft dynamics. In this work, the linear models of the pitch angle control policy and roll angle control policy come from [43] and [44], respectively. The control signals are calculated by linear feedback as follows:

$$\begin{bmatrix} \delta_a^{cd} \\ \delta_r^{cd} \end{bmatrix} = - \begin{bmatrix} 28.015 & -9.750 & -8.384 & -6.112 \\ -0.722 & 1.258 & 1.371 & -9.192 \end{bmatrix} x_\theta$$

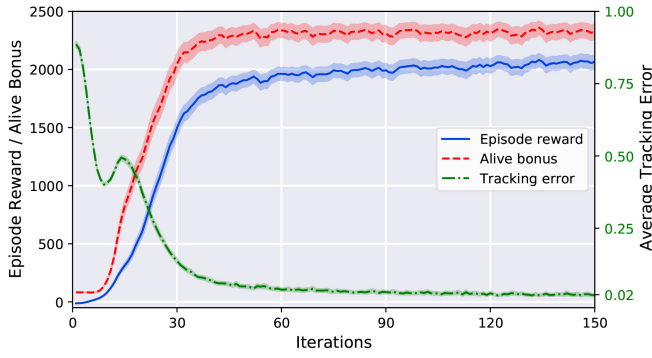


Fig. 7. Learning curves of the episode reward (blue), alive bonus (red), and tracking error (green) of the final control policy.

$$\delta_e^{cd} = -[-17.287 \quad 0.001 \quad -5.312 \quad -28.049]x_\varphi \quad (19)$$

where $x_\theta = [\tilde{\theta} - \theta, vt, \alpha, Q]$ and $x_\varphi = [\beta, \tilde{\varphi} - \varphi, P, R]$ are the states for the roll and pitch control policies, respectively.

B. Results

As shown in Fig. 6, we train the control policy with all pairs of coefficients, and test each control policy with 64 random target signals. The evaluation metrics in Fig. 6(a) and (b) are the alive bonus and tracking error, respectively. The scores are normalized by their maximum and minimum values. A higher score means that the control policy can better keep aircraft stability or track the target signals. Fig. 6 shows that different coefficient combinations lead to differences in performance. Higher c_1 may lead to the instability of aircraft because the control policy focuses more on the tracking error and neglects to keep stability. Appropriate c_2 (0.05) can help the control policy keep aircraft stability. Much high c_2 may lead to poor control performance due to the much small actions.

According to the evaluation results, we first filter control policies based on the condition that the alive bonus score is greater than 0.9. Among the filtered control policies, we select the one with the best tracking performance as the final control policy, whose training coefficients are $(c_1, c_2) = (1.5, 0.05)$. The learning curves of this control policy is shown in Fig. 7, which is obtained by averaging over three repeated experiments. The curves represent the mean values and the shaded parts represent the confidence intervals. The blue curve indicates that the episode reward rises rapidly at the beginning of the training and then converges. The red curve and green curve represent the alive bonus and average tracking error of each episode, respectively. The two curves show that stability and tracking performance increase with the number of training iterations. For the alive bonus curve, since we set the maximum timesteps of an episode as 2500 (100 s), the result indicates that the converged control policy can stabilize the aircraft well. In the tracking error curve, the tracking error is the sum of roll and pitch errors. The converged value 0.02 means that the control policy can keep an average tracking error of 1° .

Fig. 8 shows the tracking results of three control policies under the target signals described in the last section. All

control policies can manipulate the aircraft to track the target signals. The PPO-based control policy has less overshoot, faster response, and lower steady-state error than PID-based and LQR-based control policies. The PID-based control policy has obvious oscillation, and the LQR-based control policy has obvious delay. The PPO-based control policy can track the target signal quickly and smoothly, especially the step signal and random signal. Furthermore, the proposed algorithm can easily find a 6-DOF control policy with better performance and does not require any complicated analysis of the aircraft dynamics like LQR and other traditional control methods.

VII. EXPERIMENTS ON AIR COMBAT POLICY

A. Experimental Setup

In the fictitious self-play training, we train each generation combat policy to combat against the former generations. We use the *level straight flight* policy as the first policy to initialize the historical combat policy pool. This simple policy keeps the initial altitude of the aircraft and flies at the same direction without other maneuvers. We end one generation when the average reward does not increase significantly for more than 50 iterations, and the deviation is less than 10. Then, we assume that the obtained policy is the approximate best response of the former generations. When visualizing the combat trajectories of combat policies, we consider three scenarios shown in Fig. 9. They are *face to face*, *back to back*, and *parallel reverse*, where both aircraft are in the same initial situation.

For comparison, we design a search-based combat policy as a baseline. It borrows the idea from the minimax-based policy [28] and uses our hierarchical framework to decrease decision-making difficulty. It is assumed to be aware of the opponent's combat policy to simplify the decision-making process and improve its combat ability. We discretize the continuous action space into N candidate actions, resulting in a N^3 discrete action space. These candidate actions will be combined with the predicted opponent action to simulate the future states. It finally outputs the action with the highest score, which is calculated by the score function from [4]

$$\text{Score} = \frac{\mu(\pi - |\omega_a|)^2 + (1 - \mu)|\omega_o|^2}{\pi^2} \in [0, 1] \quad (20)$$

where μ is a hyperparameter, which is set to 0.5 according to the suggestion given in [4]. ω_a and ω_o are ATAs of the ego and opponent aircraft, respectively. This score ranges from 0 to 1.

B. Main Results

Based on the control policy, we train the combat policy from scratch via fictitious self-play. Fig. 10 shows the learning curve of the combat policy. The vertical dashed lines break up the training curves at different generations, and we have a total of three generations of combat policies. Within a generation, the average reward increases until convergence. Since the macro reward only indicates the outcome of the combat policy, a higher reward means that it can defeat the combat policies in the historical combat policy pool with better

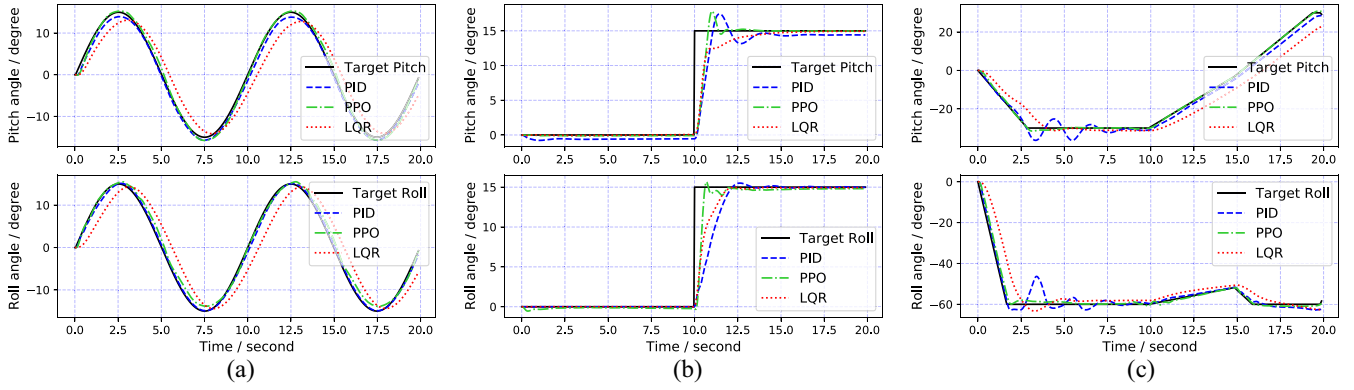


Fig. 8. Performance of trained PPO-based, PID-based, and LQR-based control policies tracking three different target signals. (a) Sinusoidal signal. (b) Step signal. (c) Random signal.

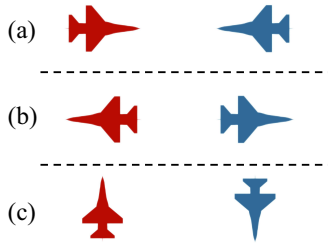


Fig. 9. Testing scenarios. (a) Face to face. (b) Back to back. (c) Parallel reverse.

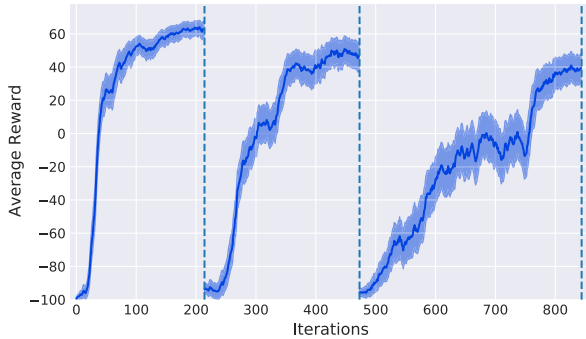


Fig. 10. Learning curves of the combat policy via fictitious self-play.

performance in various scenarios. Furthermore, the number of RL training iterations required to converge increases with the number of generations. This is because the combat policies in the pool become stronger, making it difficult for the new policy to obtain an absolute advantage.

In order to show the improvement between generations, we test the winning rates between each pair of combat policies by performing 64 combats and the results are shown in Table II. Note that the combats are randomly initialized, so even a level straight flight combat policy may win in an advantageous initial situation. As shown in Table II, higher-generation combat policies can get higher winning rates against lower-generation combat policies. It demonstrates that adding stronger combat policies generation by generation in the fictitious self-play process can further improve the performance of the learned combat policies. Fig. 11(a) shows the winning rates of the

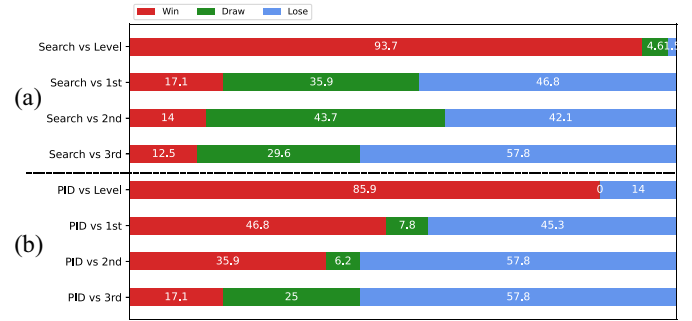


Fig. 11. Winning rates performed by (a) search-based combat policy and (b) RL-based combat policy, whose inner loop is implemented by a PID-based control policy (ablation experiment).

TABLE II
COMBAT RESULTS OF 1P FIGHTING AGAINST 2P*

2 P \ 1 P	level straight	1st-Gen	2nd-Gen
1st-Gen	96.8 / 1.6 / 1.6	-	-
2nd-Gen	96.8 / 0 / 3.2	73.4 / 6.3 / 20.3	-
3rd-Gen	96.8 / 1.6 / 1.6	71.9 / 7.8 / 20.3	73.4 / 12.5 / 14.1

* The result is the ratio of win, draw, and lose.

search-based combat policy against all generation combat policies. The search-based combat policy is powerful enough to defeat the level straight policy but is ineffective against the RL-based combat policies.

In Fig. 12, we plot the combating trajectories in the scenarios described in the last section to better demonstrate the effect of the self-play mechanism.¹ We plot the aircraft at the start and end positions to show the situation. Besides, in order to show more details of the combat, we divide the trajectories into several equal parts and mark the corresponding numbers. The same numbers on the two trajectories correspond to the same timestep. In Fig. 12, trajectories with the same color represent the same combat, and dashed trajectories represent the opponents. The results of each combat are marked in the figure.

¹ Animated demonstrations are provided at <https://sites.google.com/view/6-dof-aircombat>.

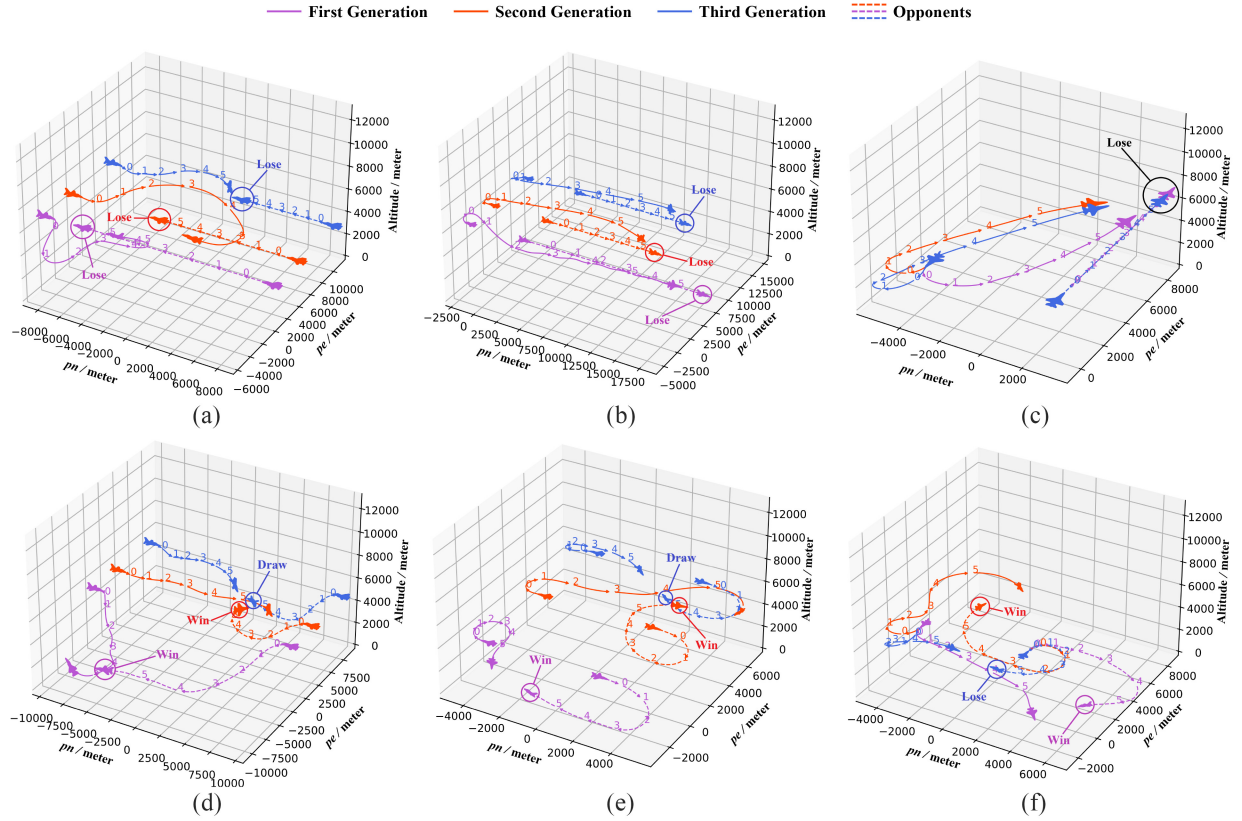


Fig. 12. Combating trajectories performed by the RL-based combat policies of all generations. (a)–(c): Trajectories (solid) are performed by the combat trajectories at different generations of fictitious self-play against the level straight flight policy (dashed). (d)–(f): Trajectories (solid) performed by all generation combat policy against the third generation combat policy (dashed). We mark the combat results of the opponents (dashed trajectories) in all figures.

- 1) *Against Level Straight Flight*: Fig. 12(a)–(c) show the combating trajectories performed by all generation combat policies against the level straight flight policy. As shown in Fig. 12(a), the first and second generations have to conduct long-distance maneuvers to avoid being attacked, and attack the opponent from the back. However, the third generation combat policy can accurately bypass the opponents' attack area and consume the least time to defeat it. In the second scenario shown in Fig. 12(b), all combat policies can quickly execute 180° turns, and combat policies with higher generation can turn faster. Fig. 12(c) also demonstrates this.
- 2) *Against Third Generation*: Fig. 12(d)–(f) show the combating trajectories performed by all generation combat policies against the third generation combat policy. In all scenarios, the third generation combat policy shows a strong aggressiveness, which tends to quickly aim its nose in the direction of the opponent. Since the former generation combat policies tend to avoid being attacked, this aggressive behavior becomes their best response by putting them in a disadvantageous state. Besides, compared with the first and second generations, the trajectories of the third generation are smoother.

C. Behavior Analysis

We show a typical maneuver in Fig. 13, where both the red and blue aircraft are controlled by the third generation combat

policy. Fig. 13(e) shows the ATAs and combat score of this trajectory from the perspective of the red aircraft. Smaller ω_a^r and larger ω_b^b result in larger score. We choose four typical frames to analyze the whole trajectory as shown in Fig. 13(a)–(d). As shown in Fig. 13(a), the red aircraft has an advantageous situation at the beginning, where its ATA is smaller than that of the blue aircraft. For the blue aircraft, it tries to narrow the ATA gap with the red aircraft and gets the smallest combat score for the red aircraft in Fig. 13(b). Both aircraft have similar ATAs, so they try to accumulate advantages through the spiral maneuver. Then, they keep narrowing their turning radius, trying to get behind their opponents. At the end of the spiral maneuver, the red aircraft accumulates enough advantages to almost target the blue aircraft in its WEZ as shown in Fig. 13(c). Due to the disadvantage of the blue aircraft, it decides to change its tactics and evade the red aircraft instead of chasing it. However, it is eventually shot down by the red aircraft as shown in Fig. 13(d).

D. Decision Experiment

We also investigate the effect of decision frequency on the performance of combat policies. As shown in Fig. 14, both sides use the third generation combat policy. The dashed trajectories are performed by the policy whose decision interval is 0.25 s. The solid trajectories are performed by the combat policies whose decision intervals are 0.1 s (purple), 0.25 s (red), 1.0 s (blue), and 4.0 s (green). When the decision interval is

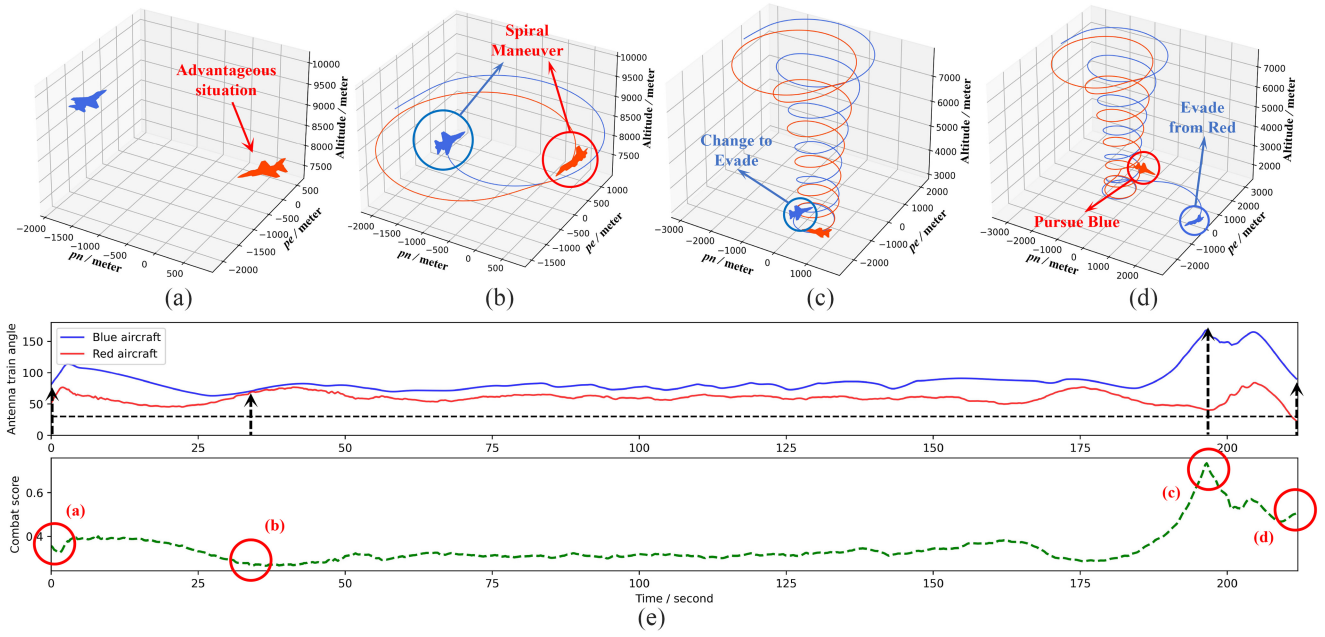


Fig. 13. Detail of a trajectory performed by the third generation combat policy. (a) Initial situation that the red aircraft has advantage. (b) Both sides are performing a spiral maneuver. (c) Blue aircraft is in danger, so it changes tactics and chooses to evade from the red aircraft. (d) Blue aircraft is eventually shot down. (e) Combat score and the ATAs during this combat.

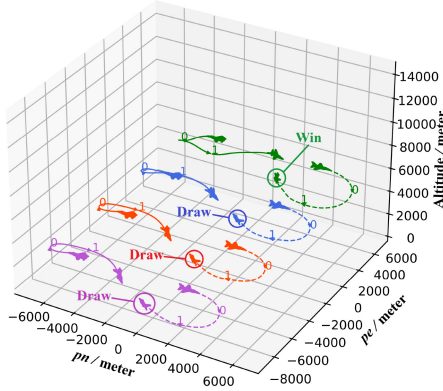


Fig. 14. Trajectories performed by the third generation combat policy with different decision intervals. The decision intervals of dashed trajectories are always 0.25 s. The decision intervals of solid are 0.1 s (purple), 0.25 s (red), 1.0 s (blue), and 4.0 s (green).

0.1, 0.25, and 1.0 s, both sides reach a draw with mutual destruction. When the decision interval is further increased to 4.0 s, the aircraft is shot down by its opponent. This is because the policy is unable to perform a timely response at some critical timesteps, making it continuously lose its advantage. The results show that the much smaller decision frequency (high decision interval) may lead to a decrease in policy performance.

E. Ablation Experiment

In the ablation experiments, we present two parts to illustrate the effect of our method.

- 1) *Replace the RL-Based Control Policy:* In the proposed framework, the combat policy depends on the RL-based control policy, which has better tracking performance

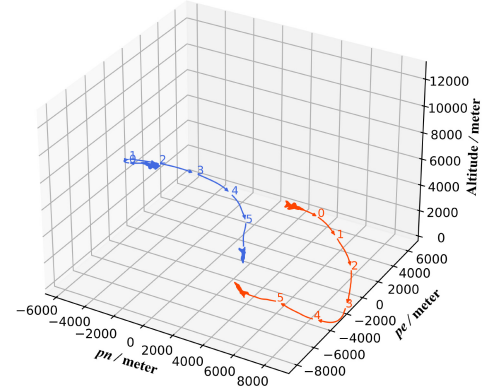


Fig. 15. Trajectories performed by the combat policy with RL-based control policy (blue) and PID-based control policy (red).

than PID-based control policy. In this experiment, we replace the RL-based control policy with the PID-based control policy to test their winning rates. The third generation combat policy with the RL-based control policy fights against all combat policies with the PID-based control policy as shown in the second part of Fig. 11(b). We also get these winning rates by performing 64 random combats. The winning rate of the combat policy with PID-based control policy is much lower than that with RL-based control policy. Besides, it still has some winning rate against these combat policies, indicating that the proposed combat policy can be integrated with other control policies to achieve intelligent control. Fig. 15 shows the trajectories of one back to back combat. The blue trajectory is performed by the combat policy with RL-based control policy, while the red trajectory is performed by the combat policy

TABLE III
PARAMETER SETTINGS OF THE CONTROL POLICY AND COMBAT POLICY

Setting	Name	Control Policy	Combat Policy
Training Settings	Worker number	64	
	PPO clip parameter	0.2	
	GAE parameter λ	0.95	
	Iteration per epoch	20	5
	Buffer size	320000	51200
	Batch size	32000	5120
	Decision interval	0.04 seconds	0.25 seconds
Network Settings	Discount factor γ	0.99	0.95
	Network neurons	256×2	
	Activation function	ReLU	
	Optimizer	SGD	
	Learning rate	0.0001	0.0005

with PID-based control policy. The turning speed of the red aircraft is obviously lower than the blue aircraft, which leads to its failure. The result indicates that the RL-based control policy can better execute the macro commands from the combat policy, thus gaining more combat advantages.

- 2) *Remove Fictitious Self-Play Mechanism*: The first generation combat policy is actually the result of the purely PPO training experiment, which is an ablation experiment to remove the fictitious self-play training. The higher generation combat policies can achieve more than 75% winning rate against it. This result shows that combat policies trained by fictitious self-play mechanism can achieve better performance than those trained by simple RL algorithms.

VIII. CONCLUSION

This article presents a novel training method for WVR air-to-air combat under 6-DOF dynamics. We propose a hierarchical framework to reduce the difficulty of decision making. We present a two-stage training mechanism to train the inner loop control policy and the outer combat policy separately. Experiment results demonstrate the performance of the control policy and combat policy. The control policy can control the aircraft to track several kinds of target signals quickly and accurately, and the combat policy can learn powerful tactics to defeat opponents. We also demonstrate the effect of the evolution between different generations and analyze the learned maneuver behaviors of the final combat policy. Results show that the proposed method can address the difficult 6-DOF air combat problem. Besides, our method can also help to solve other continuous action space zero-sum games. Future works could include general-sum games like air combats between more than one aircraft and safe-RL-based methods.

APPENDIX PARAMETER SETTINGS

We use the PPO codes implemented by RLlib [48] for training. Table III shows the parameter settings of the control policy and the combat policy in detail. We use 64 workers to perform simulations simultaneously to collect training data. When the replay buffer is full, the algorithm conducts several

iterations (20 or 5) to update the network parameters. In each iteration, the algorithm samples multiple batches from the full buffer until all data are sampled in a round.

REFERENCES

- [1] S. Kumar, "A brief review on unmanned combat aerial vehicle," *SSRN Electron. J.*, vol. 5, no. 23, p. 7, 2020.
- [2] J. Jordan, "The future of unmanned combat aerial vehicles: An analysis using the three horizons framework," *Futures*, vol. 134, Dec. 2021, Art. no. 102848.
- [3] J. Hu, L. Wang, T. Hu, C. Guo, and Y. Wang, "Autonomous maneuver decision making of dual-UAV cooperative air combat based on deep reinforcement learning," *Electronics*, vol. 11, no. 3, p. 467, 2022.
- [4] H. Shin, J. Lee, H. Kim, and D. H. Shim, "An autonomous aerial combat framework for two-on-two engagements based on basic fighter maneuvers," *Aerosp. Sci. Technol.*, vol. 72, pp. 305–315, Jan. 2018.
- [5] Y.-F. Li, J.-P. Shi, W. Jiang, W.-G. Zhang, and Y.-X. Lyu, "Autonomous maneuver decision-making for a UCAV in short-range aerial combat based on an MS-DDQN algorithm," *Defence Technol.*, vol. 18, no. 9, pp. 1697–1714, 2022.
- [6] R. Isaacs, "Games of pursuit," RAND Corp., Santa Monica, CA, USA, Scientific Rep., 1951. [Online]. Available: <https://www.rand.org/pubs/papers/P257.html>.
- [7] Ö. F. Arar and K. Ayan, "A flexible rule-based framework for pilot performance analysis in air combat simulation systems," *Turkish J. Electr. Eng. Comput. Sci.*, vol. 21, no. 8, pp. 2397–2415, 2013.
- [8] A. R. Chappell, "Knowledge-based reasoning in the paladin tactical decision generation system," in *Proc. 11th IEEE/AIAA Digit. Avionics Syst. Conf.*, 1992, pp. 155–160.
- [9] N. Rao, S. Kashyap, G. Gopalaratnam, and D. Mandal, "Situation and threat assessment in BVR combat," in *Proc. AIAA Guid. Navig. Control Conf.*, 2011, p. 6241.
- [10] J. S. McGrew, J. P. How, B. Williams, and N. Roy, "Air-combat strategy using approximate dynamic programming," *J. Guid. Control Dyn.*, vol. 33, no. 5, pp. 1641–1654, 2010.
- [11] K. Shao, Z. Tang, Y. Zhu, N. Li, and D. Zhao, "A survey of deep reinforcement learning in video games," 2019, *arXiv:1912.10944*.
- [12] Y. Zhu and D. Zhao, "Online minimax Q network learning for two-player zero-sum Markov games," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 3, pp. 1228–1241, Mar. 2022.
- [13] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [14] Z. Tang, Y. Zhu, D. Zhao, and S. M. Lucas, "Enhanced rolling horizon evolution algorithm with opponent model learning: Results for the fighting game AI competition," *IEEE Trans. Games*, vol. 15, no. 1, pp. 5–15, Mar. 2023.
- [15] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [16] K. Shao, Y. Zhu, and D. Zhao, "StarCraft micromanagement with reinforcement learning and curriculum transfer learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 3, no. 1, pp. 73–84, Feb. 2019.
- [17] J. Chai et al., "UNMAS: Multiagent reinforcement learning for unshaped cooperative scenarios," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 2093–2104, Apr. 2023.
- [18] Y. Zhu, W. Li, M. Zhao, J. Hao, and D. Zhao, "Empirical policy optimization for n -player Markov games," *IEEE Trans. Cybern.*, early access, Jun. 24, 2022, doi: [10.1109/TCYB.2022.3179775](https://doi.org/10.1109/TCYB.2022.3179775).
- [19] Y. Zhang, B. Zhao, D. Liu, and S. Zhang, "Event-triggered control of discrete-time zero-sum games via deterministic policy gradient adaptive dynamic programming," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 8, pp. 4823–4835, Aug. 2022.
- [20] W. Li, Y. Zhu, and D. Zhao, "Missile guidance with assisted deep reinforcement learning for head-on interception of maneuvering target," *Complex Intell. Syst.*, vol. 8, no. 2, pp. 1205–1216, 2022.
- [21] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [22] Q. Yang, J. Zhang, G. Shi, J. Hu, and Y. Wu, "Maneuver decision of UAV in short-range air combat based on deep reinforcement learning," *IEEE Access*, vol. 8, pp. 363–378, 2019.
- [23] J. Xu, Q. Guo, L. Xiao, Z. Li, and G. Zhang, "Autonomous decision-making method for combat mission of UAV based on deep reinforcement learning," in *Proc. IEEE 4th Adv. Inf. Technol. Electron. Autom. Control Conf.*, vol. 1, 2019, pp. 538–544.
- [24] X. Ma, L. Xia, and Q. Zhao, "Air-combat strategy using deep Q-learning," in *Proc. Chin. Autom. Congr.*, 2018, pp. 3952–3957.

- [25] W. Yuan, Z. Xiwen, Z. Rong, T. Shangqin, Z. Huan, and D. Wei, "Research on UCAV maneuvering decision method based on heuristic reinforcement learning," *Comput. Intell. Neurosci.*, vol. 2022, Mar. 2022, Art. no. 1477078.
- [26] L. Li, Z. Zhou, J. Chai, Z. Liu, Y. Zhu, and J. Yi, "Learning continuous 3-DOF air-to-air close-in combat strategy using proximal policy optimization," in *Proc. IEEE Conf. Games (COG)*, 2022, pp. 616–619.
- [27] D. Seto, E. Ferreira, and T. Marz, "Case study: Development of a baseline controller for automatic landing of an F-16 aircraft using linear matrix inequalities," Dept. Softw. Eng. Inst., Carnegie Mellon Univ., Pittsburgh, PA, USA, Rep. CMU/SEI-99-TR-020, 2000.
- [28] H. Park, B.-Y. Lee, M.-J. Tahk, and D.-W. Yoo, "Differential game based air combat maneuver generation using scoring function matrix," *Int. J. Aeronaut. Space Sci.*, vol. 17, no. 2, pp. 204–213, 2016.
- [29] Z. Wang, H. Li, H. Wu, and Z. Wu, "Improving maneuver strategy in air combat by alternate freeze games with a deep reinforcement learning algorithm," *Math. Problems Eng.*, vol. 2020, Jun. 2020, Art. no. 7180639.
- [30] W. Kong, D. Zhou, K. Zhang, and Z. Yang, "Air combat autonomous maneuver decision for one-on-one within visual range engagement base on robust multi-agent reinforcement learning," in *Proc. IEEE 16th Int. Conf. Control Autom. (ICCA)*, 2020, pp. 506–512.
- [31] Z. Yang, K. Merrick, L. Jin, and H. A. Abbass, "Hierarchical deep reinforcement learning for continuous action control," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5174–5184, Nov. 2018.
- [32] M. Wang, L. Wang, T. Yue, and H. Liu, "Influence of unmanned combat aerial vehicle agility on short-range aerial combat effectiveness," *Aerosp. Sci. Technol.*, vol. 96, Jan. 2020, Art. no. 105534.
- [33] P. Heidlauf, A. Collins, M. Bolender, and S. Bak, "Verification challenges in F-16 ground collision avoidance and other automated maneuvers," in *Proc. ARCH@ ADHS*, 2018, pp. 208–217.
- [34] S. A. Snell, D. F. Enns, and W. L. Garrard Jr., "Nonlinear inversion flight control for a supermaneuverable aircraft," *J. Guid. Control Dyn.*, vol. 15, no. 4, pp. 976–984, 1992.
- [35] D. Zhou and Q. Geng, "Multi-model and fuzzy PID control for fixed-wing UAV," in *Proc. 3rd Int. Conf. Mechatronics Robot. Autom.*, 2015, pp. 523–528.
- [36] B. Porter and D. L. Hicks, "Genetic design of unconstrained digital PID controllers," in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, vol. 1, 1995, pp. 478–485.
- [37] L. Li, L. Song, T. Li, and J. Fu, "Event-triggered output regulation for networked flight control system based on an asynchronous switched system approach," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 12, pp. 7675–7684, Dec. 2021.
- [38] A. Abbaspour, K. K. Yen, P. Forouzaneshad, and A. Sargolzaei, "A neural adaptive approach for active fault-tolerant control design in UAV," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 9, pp. 3401–3411, Sep. 2020.
- [39] C.-M. Lin and E.-A. Boldbaatar, "Autoland control using recurrent wavelet Elman neural network," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 45, no. 9, pp. 1281–1291, Sep. 2015.
- [40] A. Ashraf, W. Mei, L. Gaoyuan, Z. Anjum, and M. M. Kamal, "Design linear feedback and LQR controller for lateral flight dynamics of F-16 aircraft," in *Proc. Int. Conf. Control Autom. Inf. Sci. (ICCAIS)*, 2018, pp. 367–371.
- [41] A. P. Pope et al., "Hierarchical reinforcement learning for air-to-air combat," in *Proc. Int. Conf. Unmanned Aircraft Syst.*, 2021, pp. 275–284.
- [42] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, vol. 80, Jul. 2018, pp. 1861–1870.
- [43] W. Ahmed, "System modeling and controller design for lateral and longitudinal motion of F-16," *Autom. Control Intell. Syst.*, vol. 7, pp. 39–45, Feb. 2019.
- [44] B. L. Stevens, F. L. Lewis, and E. N. Johnson, *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Hoboken, NJ, USA: Wiley, 2015.
- [45] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [46] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [47] G. W. Brown, "Iterative solution of games by fictitious play," *Act. Anal. Prod. Allocation*, vol. 13, no. 1, pp. 374–376, 1951.
- [48] E. Liang et al., "RLlib: Abstractions for distributed reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 3053–3062.



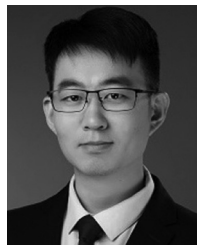
Jiajun Chai (Graduate Student Member, IEEE) received the B.S. degree in automation from the Faculty of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an, China, in 2020. He is currently pursuing the Ph.D. degree in control theory and control engineering with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China.

His current research interests include multiagent reinforcement learning, deep learning, and game AI.



Wenzhang Chen received the B.S. degree in computer science from the University of Science and Technology of China, Hefei, China, in 2022. He is currently pursuing the master's degree in control theory and control engineering with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China.

His current research interests include multiagent reinforcement learning, deep learning, and game AI.



Yuanheng Zhu (Senior Member, IEEE) received the B.S. degree in automation from Nanjing University, Nanjing, China, in 2010, and the Ph.D. degree in control theory and control engineering from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2015.

From 2015 to 2017, he was an Assistant Professor with the Institute of Automation, Chinese Academy of Sciences, where he is currently an Associate Professor. From 2017 to 2018, he was a Visiting Scholar with the Department of Electrical, Computer, and Biomedical Engineering, University of Rhode Island, Kingston, RI, USA. His current research interests include deep reinforcement learning, game theory, game intelligence, and multiagent learning.

Dr. Zhu serves as an Associate Editor for the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.



Zong-Xin Yao received the Ph.D. degree in measuring and testing technologies and instruments from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2004.

From 2004 to 2006, he worked as a Postdoctoral Researcher of Navigation, Guidance, and Control with Beihang University, Beijing, China. He is currently a Researcher and a Senior Designer with Shenyang Aircraft Design and Research Institute, Shenyang, Liaoning, China, and a top-level expert in the field of artificial intelligence with China Aviation Industry Corporation, Beijing. He mainly engages in theoretical methods and application technologies research in unmanned aerial vehicle system design, information fusion, intelligent decision making, and collaborative control. He has profound expertise in the field of intelligent game and control of unmanned aerial vehicle systems.



Dongbin Zhao (Fellow, IEEE) received the Ph.D. degree in material processing engineering from the Harbin Institute of Technology, Harbin, China, in 2000.

He has been a Professor with the Institute of Automation, Chinese Academy of Sciences, Beijing, China, since 2002, and also a Professor with the University of Chinese Academy of Sciences, Beijing. He has authored or coauthored six books and over 100 international journal articles. His current research interests include deep reinforcement learning, computational intelligence, autonomous driving, game artificial intelligence, robotics, and smart grids.

Prof. Zhao works as a guest editor for several renowned international journals. He serves as an Associate Editor for the IEEE TRANSACTIONS ON CYBERNETICS and *IEEE Computation Intelligence Magazine*.